Projet de fin de la partie : Cloud Intelligence

Énoncé du projet

Le projet est composé de **quatre parties**. Les parties 2, 3 et 4 utilisent la même installation OpenStack ; la partie 1 (CloudSim Plus) est indépendante et peut être faite en parallèle.

Attention : les installations doivent se faire **obligatoirement sur votre machine** (une VM locale, par exemple VirtualBox ou VMware), et **pas dans une VM externe** (Azure, AWS, Google Cloud ou tout autre cloud).

Partie 1. Simulation d'un cloud universitaire avec CloudSim Plus

Proposer une architecture Cloud Computing pour l'**Université Abdelmalek Essaâdi**, puis la **simuler avec CloudSim Plus**.

1. **Se documenter** sur l'université : nombre d'établissements et de villes, nombre d'étudiant(e)s et d'enseignants, services numériques utilisés ou potentiels (plateforme e-learning, inscriptions, messagerie, examens en ligne, bibliothèque numérique…). Citer vos sources.
2. **Proposer une architecture** : nombre et emplacement des datacenters, nombre et capacité des serveurs (hôtes), machines virtuelles par service, modèle de service (IaaS, PaaS, SaaS) et de déploiement (cloud privé, public, hybride). Justifier par un schéma.
3. **Modéliser les charges** : nombre de connexions et types de requêtes (cloudlets) pour une période normale et pour une **période de pointe** (inscriptions, examens).
4. **Simuler** les deux périodes dans CloudSim Plus et comparer au moins deux politiques : ordonnancement des tâches (`CloudletSchedulerTimeShared` / `CloudletSchedulerSpaceShared`) ou allocation des VM (`VmAllocationPolicySimple` / `VmAllocationPolicyBestFit`). Ajouter des VM automatiquement en période de pointe (`HorizontalVmScaling`).
5. **Analyser** les résultats : temps de réponse, temps d'exécution, taux d'utilisation des hôtes, coût. Conclure : l'architecture proposée suffit-elle en période de pointe?

À rendre : le schéma de l'architecture, le code Java de la simulation, un tableau des résultats et leur analyse.

Partie 2. OpenStack: IaaS et SaaS

1. **Vérifier l'installation DevStack** : accès au tableau de bord Horizon, puis en ligne de commande `openstack service list`, `openstack compute service list`, `openstack network agent list`. Expliquer en une phrase le rôle de chaque service listé (Keystone, Nova, Glance, Neutron, Placement, Cinder…).
2. **IaaS avec CirrOS** : créer un réseau privé, un routeur et un groupe de sécurité (SSH et ICMP autorisés), une paire de clés, puis lancer une instance **CirrOS** et lui associer une **IP flottante**. Tester avec `ping` puis en **SSH** avec **PuTTY** (ou `ssh`).
3. **SaaS** : dans une instance **Ubuntu Server** (image cloud), installer et rendre accessible une application de votre choix (par exemple Nextcloud, WordPress, un wiki, une application Flask…). Montrer l'application ouverte dans le navigateur.

Conseil : l'instance SaaS a besoin de 1 à 2 Go de RAM. Créez un gabarit (flavor) adapté, par exemple `openstack flavor create --ram 1536 --disk 10 --vcpus 1 m1.saas`, et arrêtez les instances dont vous n'avez plus besoin.

À rendre : les commandes utilisées et les captures d'écran (Horizon, topologie réseau, connexion PuTTY, application SaaS).

Partie 3. Infrastructure as Code : Terraform et Ansible

1. Avec **Terraform** (fournisseur `terraform-provider-openstack`), décrire puis créer une instance **Ubuntu** sous OpenStack, avec son groupe de sécurité (SSH et HTTP) et son IP flottante. Utiliser `terraform init`, `plan`, `apply`, puis `destroy` à la fin.
2. Avec **Ansible**, sur cette instance : installer **Nginx**, démarrer et activer le service, et déployer une page web qui affiche **votre nom, votre numéro d'étudiant et le nom de l'instance**.
3. Expliquer la différence entre les deux outils : provisionner l'infrastructure (Terraform) et configurer les machines (Ansible).

Conseil : Ansible ne s'exécute pas directement sous Windows : lancez Terraform et Ansible depuis la VM DevStack (Linux) ou depuis WSL. Les identifiants OpenStack peuvent être lus dans `clouds.yaml`.

À rendre : les fichiers `.tf`, l'inventaire et le playbook Ansible, la sortie de `terraform apply` et une capture de la page web.

Partie 4. Service Level Agreement (SLA)

Un SLA est un contrat entre l'utilisateur et le fournisseur de services cloud. Il fixe des objectifs mesurables (disponibilité, temps de réponse…) et la façon de les mesurer. Il peut se présenter sous forme d'un fichier texte ou JSON.

1. **Rédiger un SLA** (`sla.txt` ou `sla.json`) dont l'objectif est la **disponibilité des instances OpenStack** : **99,5 %** sur une période d'évaluation **quotidienne**. Préciser : les instances concernées, la définition d'une instance « disponible », la fréquence de mesure, la formule de calcul et ce qui se passe si l'objectif n'est pas atteint.
2. **Écrire un programme Python** qui se connecte au projet OpenStack (bibliothèque `openstacksdk`, authentification Keystone) et vérifie **toutes les 5 minutes** l'état des instances (API Nova ; vous pouvez ajouter un test réseau : ping ou port SSH).
3. Le programme **enregistre chaque mesure** (date, instance, état) et **calcule la disponibilité** de chaque instance : mesures « disponible » ÷ mesures totales × 100.
4. **Provoquer une panne** pendant la surveillance (`openstack server stop` puis `start`) et montrer que le programme la détecte et que la disponibilité baisse.
5. **Ajouter le rapport de surveillance** au fichier SLA : période, nombre de mesures, disponibilité de chaque instance, objectif respecté ou non.

Conseil : surveillez 2 ou 3 petites instances CirrOS : elles ne consomment presque pas de mémoire. Si votre machine ne peut pas rester allumée 24 heures, faites une période d'évaluation plus courte (par exemple 2 heures) et expliquez dans le rapport comment le calcul s'étend à une journée. Ceilometer et Aodh ne sont pas installés par défaut dans DevStack : Nova et Keystone suffisent.

Question à traiter dans le rapport : sur une journée, avec une mesure toutes les 5 minutes, combien de mesures en échec peut-on tolérer pour rester au-dessus de 99,5 % ? Quelle durée de panne cela représente-t-il ?

À rendre : le fichier SLA avec son rapport, le script Python, le journal des mesures et la capture de la panne détectée.

Livrables

Déposer sur Google Classroom, **avant la date limite** :

- `Nom_Prenom_Rapport.pdf` : le rapport, avec pour chaque partie les étapes, les commandes, les **impressions d'écran** et vos explications ;
- `Nom_Prenom_Code.zip` : le projet Java de la simulation CloudSim Plus, les fichiers Terraform et Ansible, le fichier SLA et le script Python.

Important : Les captures d'écran doivent montrer votre nom d'utilisateur ou le nom de vos instances, et la date. Toutes les installations doivent être réalisées **sur votre machine** : l'installation d'OpenStack dans une VM d'un cloud externe n'est pas acceptée.